

FSDP (Fully Sharded Data
Parallel)

About me

- MSU finished (haha)
- разработчик в команде инфраструктуры обучения YandexGPT
- мэйнтейнер YaFSDP (по возможности)

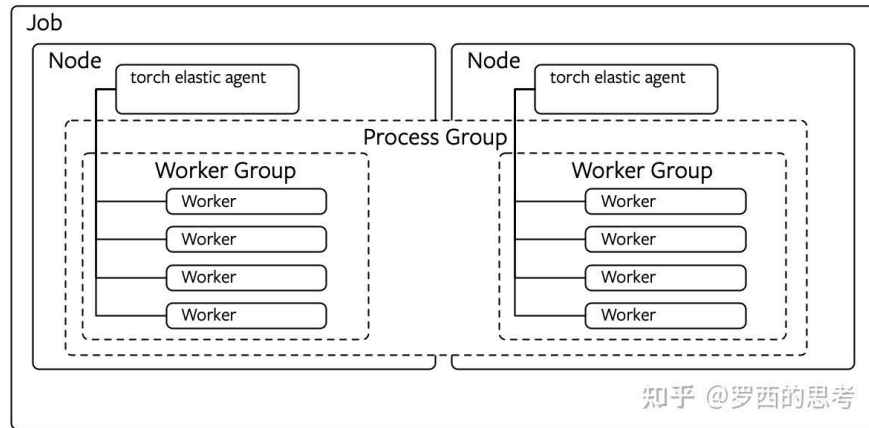


Plan

- Collective ops intro
- DDP recap
- Mixed precision recap
- FSDP basics
- Memory usage
- Communication overlap
- Sharding strategies
- HSDP
- Resharding to a smaller world size
- CPU offloading
- FSDP implementations

Collective ops intro - `torch.distributed`

- Process groups: 1 GPU - 1 process
- Distributed training requires inter-process communication ops:
 - AllReduce
 - AllGather
 - ReduceScatter
 - ...
- Backends implement these ops: NCCL for GPUs, GLOO for CPUs, etc.

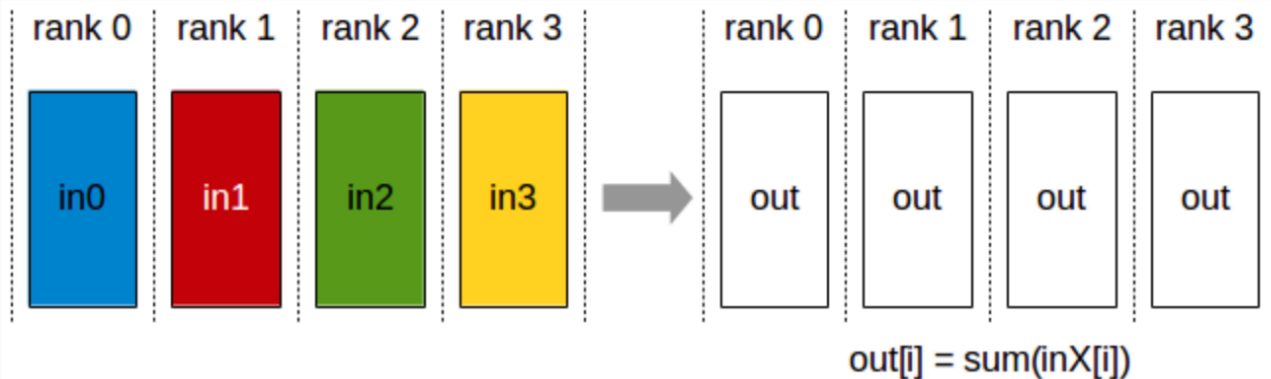


Collective ops intro - AllReduce

AllReduce

The AllReduce operation performs reductions on data (for example, sum, min, max) across devices and stores the result in the receive buffer of every rank.

In a *sum* allreduce operation between k ranks, each rank will provide an array *in* of N values, and receive identical results in array *out* of N values, where $out[i] = in0[i] + in1[i] + \dots + in(k-1)[i]$.



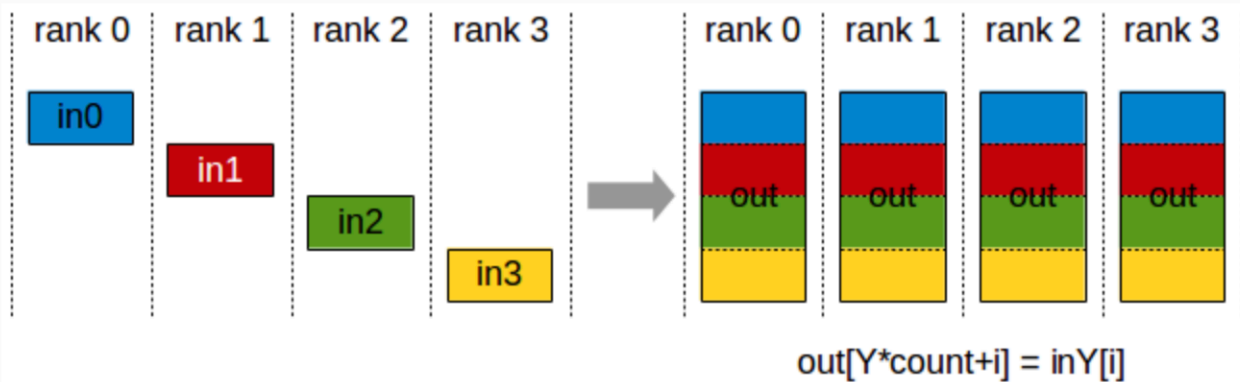
All-Reduce operation: each rank receives the reduction of input values across ranks.

Collective ops intro - AllGather

AllGather

The AllGather operation gathers N values from k ranks into an output buffer of size $k*N$, and distributes that result to all ranks.

The output is ordered by the rank index. The AllGather operation is therefore impacted by a different rank to device mapping.



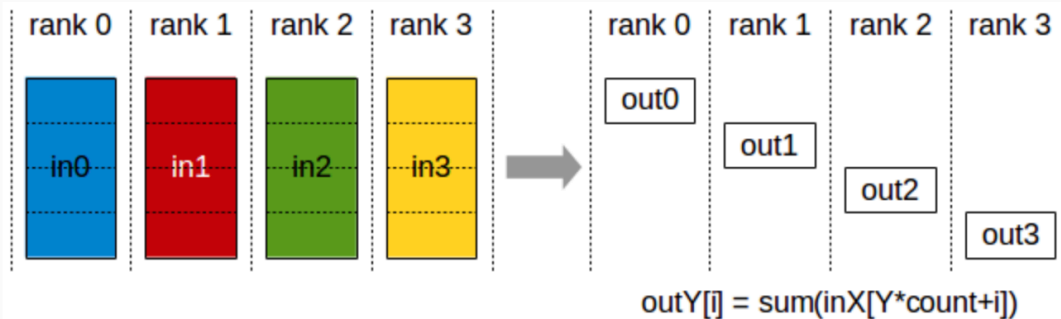
AllGather operation: each rank receives the aggregation of data from all ranks in the order of the ranks.

Collective ops intro - ReduceScatter

ReduceScatter

The ReduceScatter operation performs the same operation as Reduce, except that the result is scattered in equal-sized blocks between ranks, each rank getting a chunk of data based on its rank index.

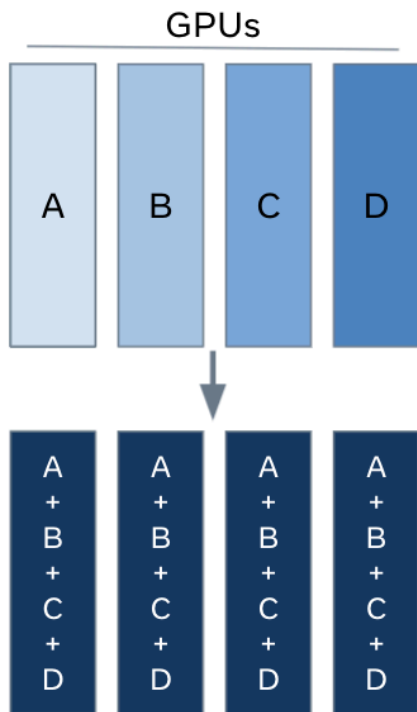
The ReduceScatter operation is impacted by a different rank to device mapping since the ranks determine the data layout.



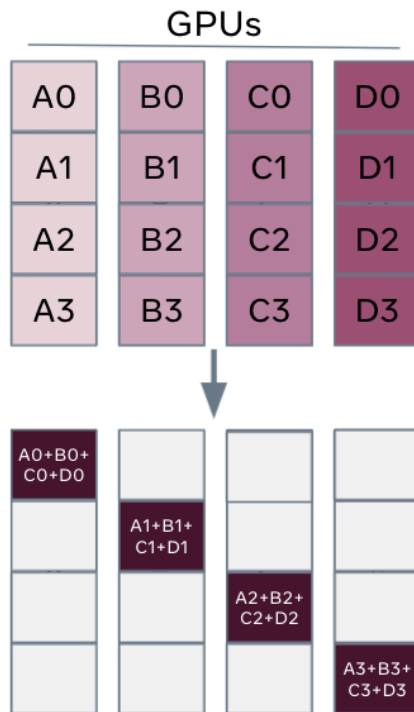
Reduce-Scatter operation: input values are reduced across ranks, with each rank receiving a subpart of the result.

Collective ops intro

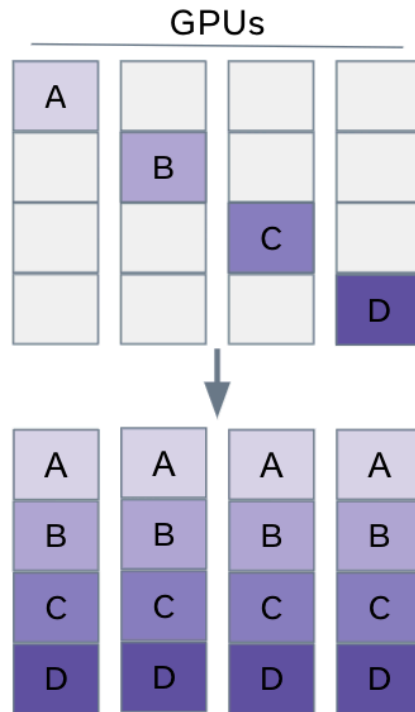
All Reduce



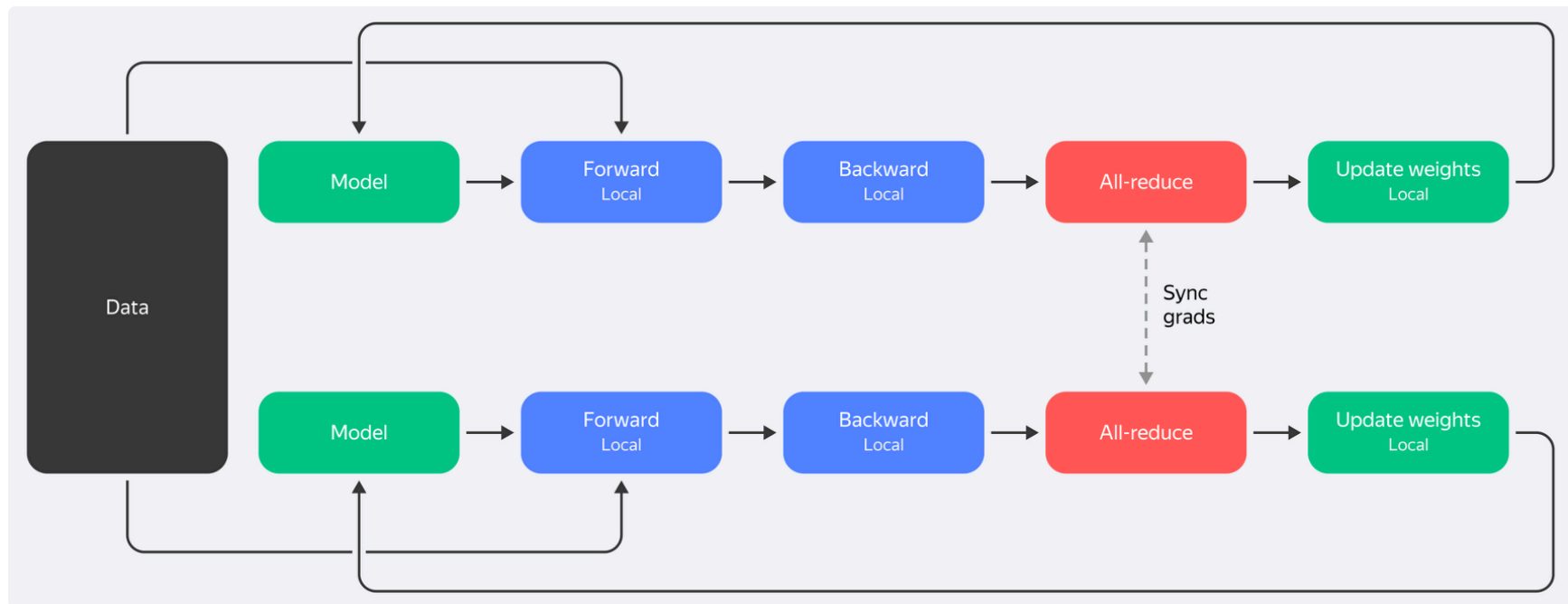
Reduce- Scatter



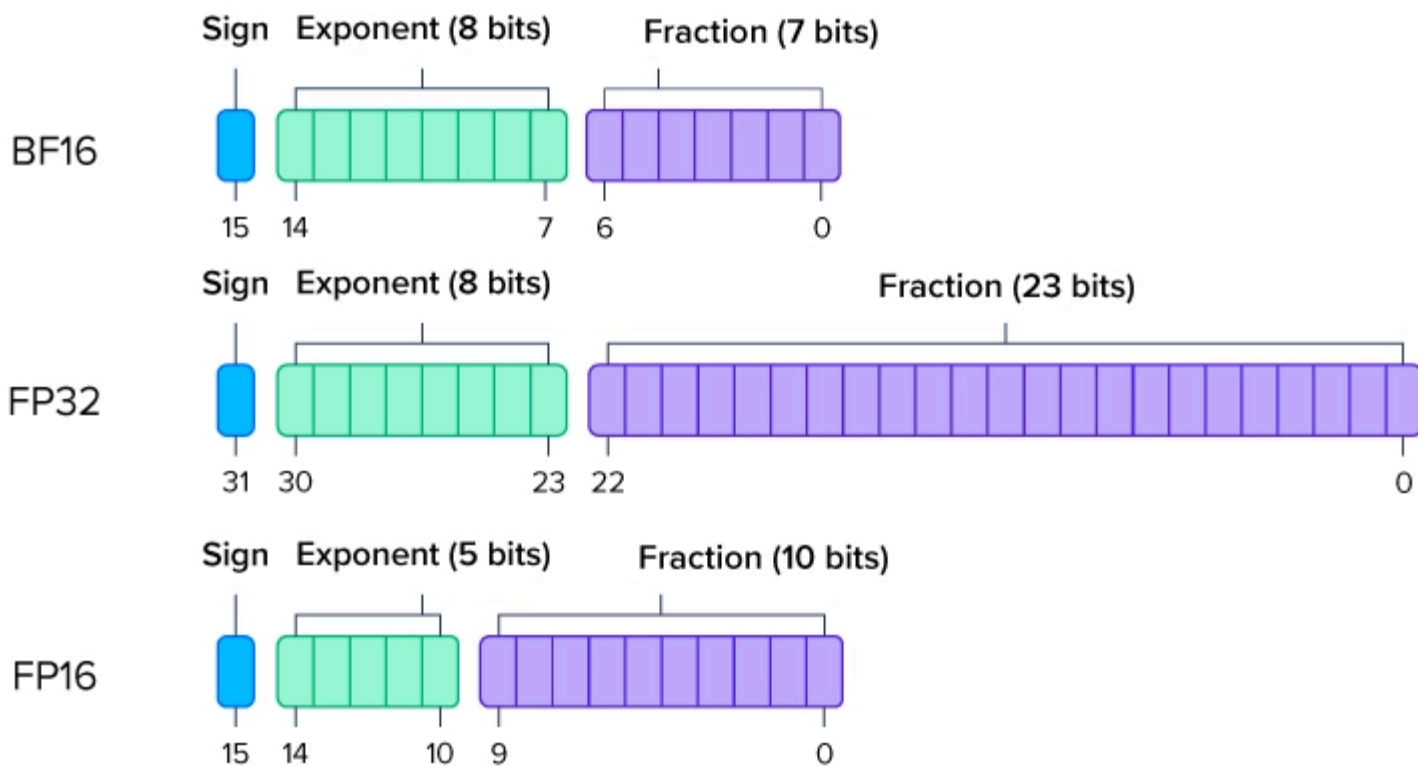
All-gather



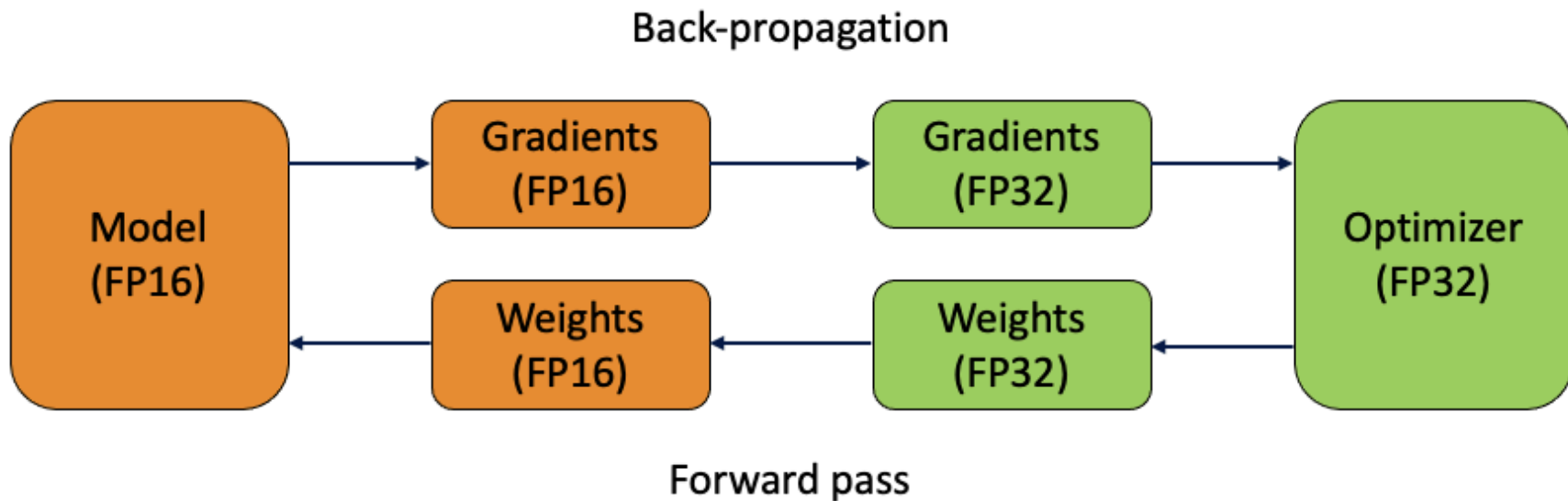
DDP recap



Mixed precision recap



Mixed precision recap



Fully Sharded Data Parallel

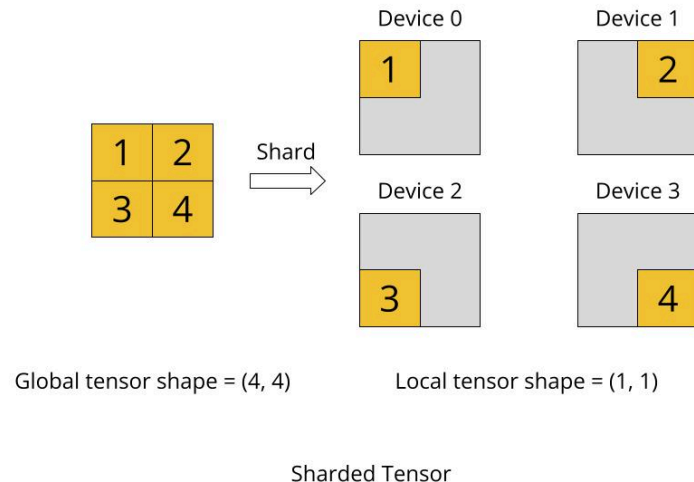
Fully SHARDED Data Parallel

Huge models don't fit into one GPU.

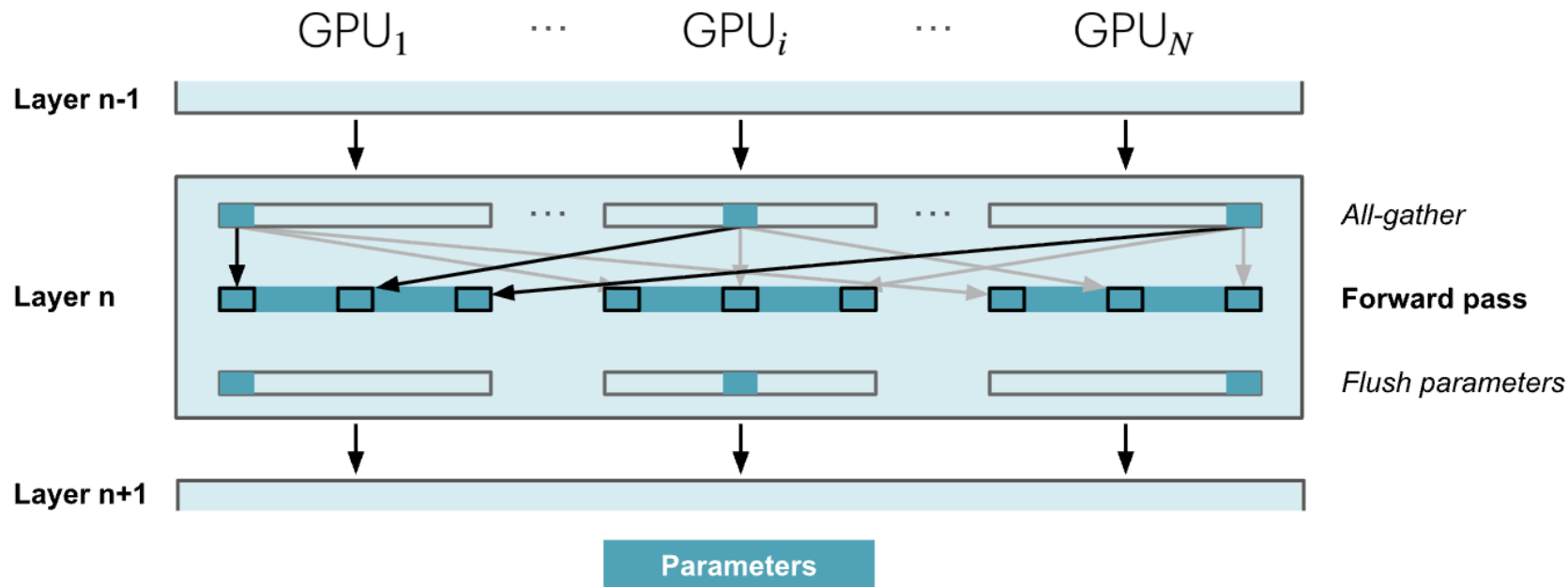
Lets shard something!

We shard:

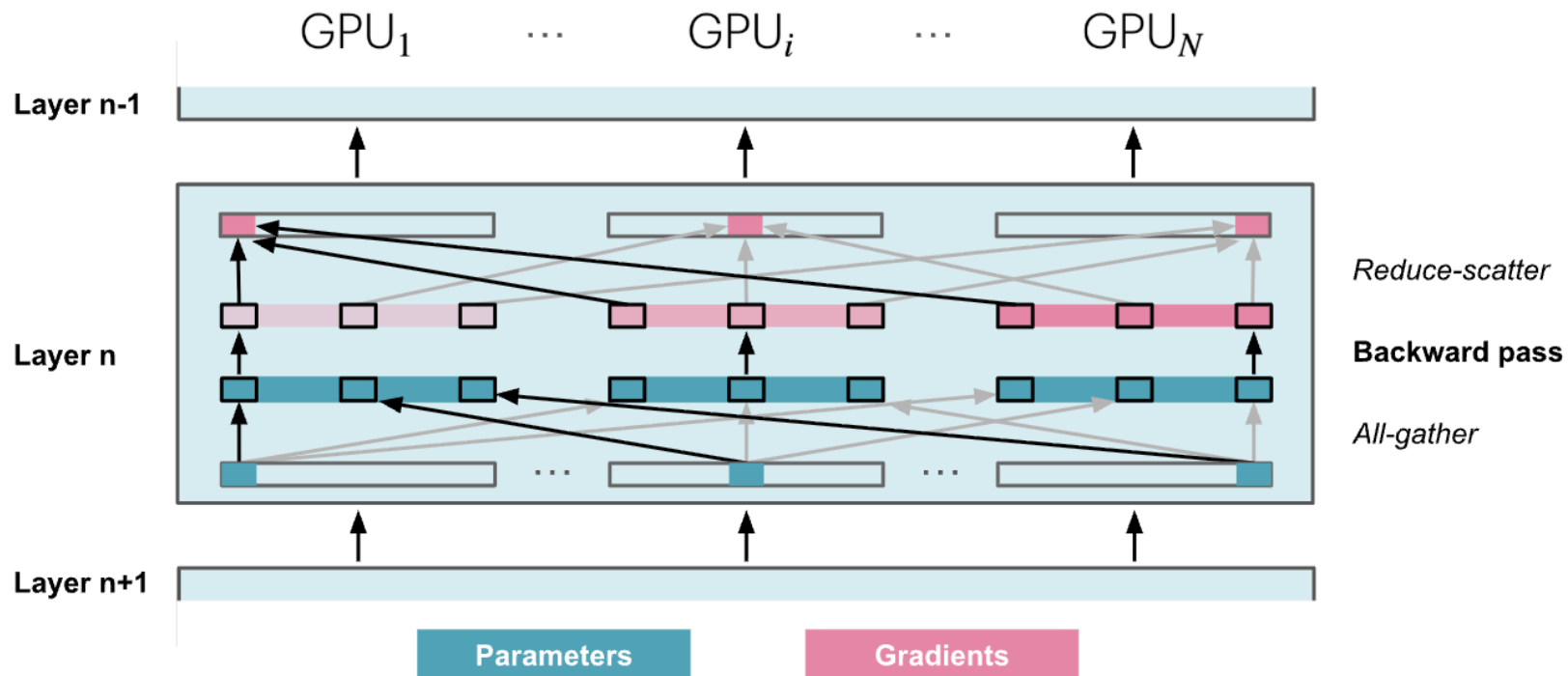
- weights
- gradients
- optimizer states



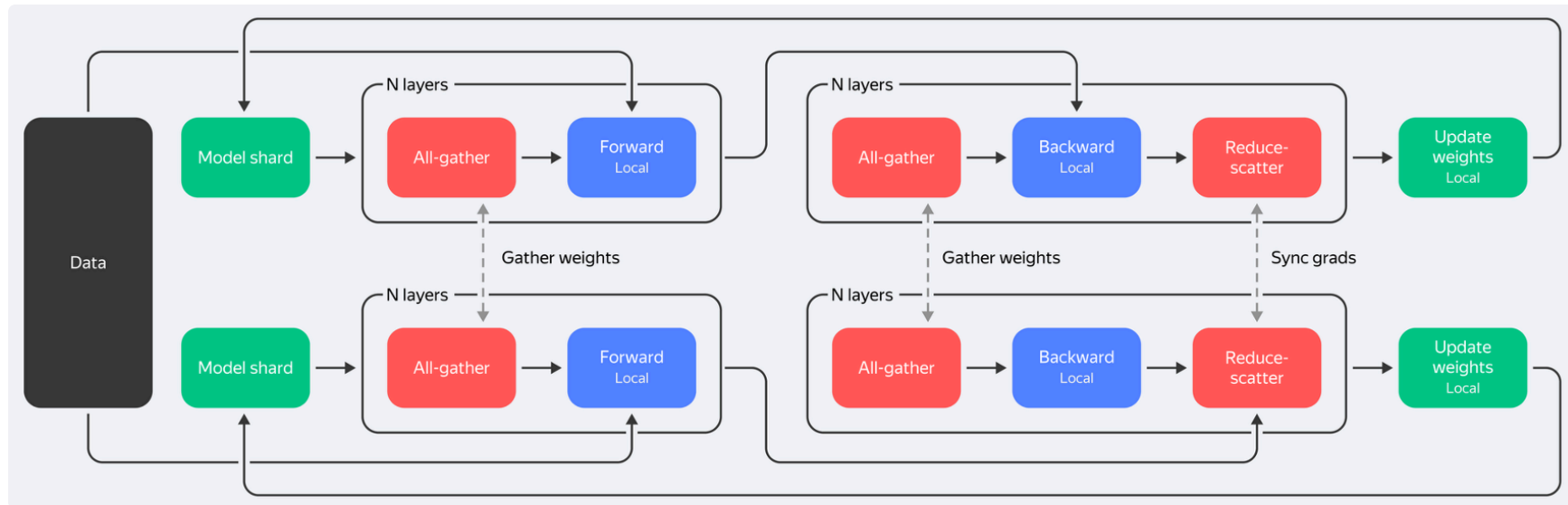
FSDP - forward



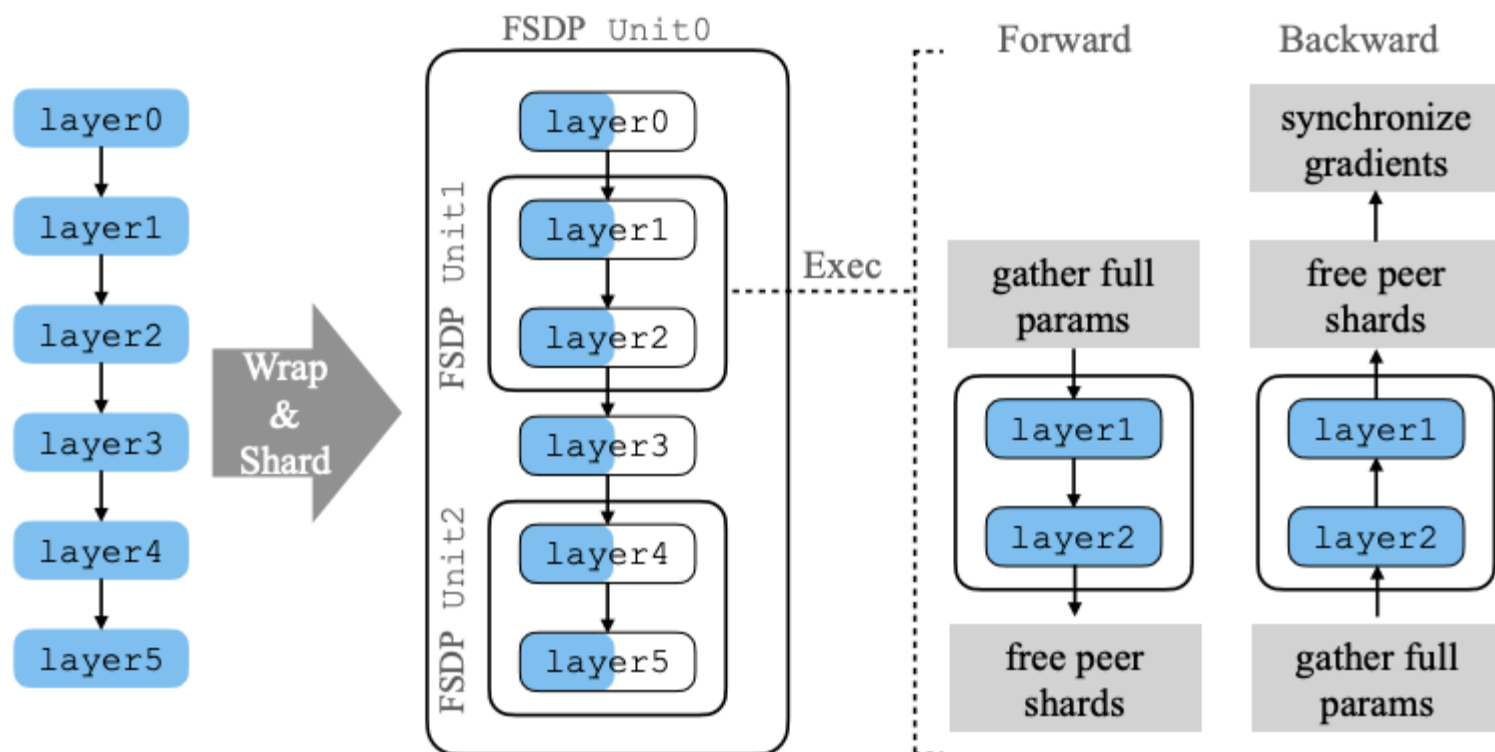
FSDP - backward



FSDP - overview



FSDPUnit



FSDP - memory usage (LLama 70B on 128 GPUs)

Assuming per-transformer-block sharding.

Params per block (80 blocks):

```
3 (w1, w2, v1) * 8192 (hidden-size) * 28672 (intermediate-size)
+ (2 (q-proj + out-proj) + 2 * 1/8 (kw-proj)) * 8192 * 8192
+ ...
~= 0.85B
```

FSDP - memory usage (LLama 70B on 128 GPUs)

Sharded states (in fp32):

```
weights = 70B / 128 * 4 bytes ~= 2.2 GB  
grads = weights  
optim_states = 2 * weights (for AdamW)
```

Unsharded states (in fp16 / bf16):

```
weights = 2 * 0.85B * 2 bytes = 3.4 GB  
grads = weights
```

Total: $(3.4 \text{ GB} * 2 + 2.2 \text{ GB} * (1 + 1 + 2)) = 15.6 \text{ GB} = 20 \% \text{ of an } 80\text{GB GPU}$

Not too bad! (compared to DDP with $4 * 70\text{B} * 4 \text{ bytes} = 1.1\text{TB}$)

FSDP - memory usage (LLama 70B on 1024 GPUs)

Sharded states (in fp32):

```
weights = 70B / 1024 * 4 bytes ~= 0.275 GB  
grads = weights  
optim_states = 2 * weights (for AdamW)
```

Unsharded states (in fp16 or bf16):

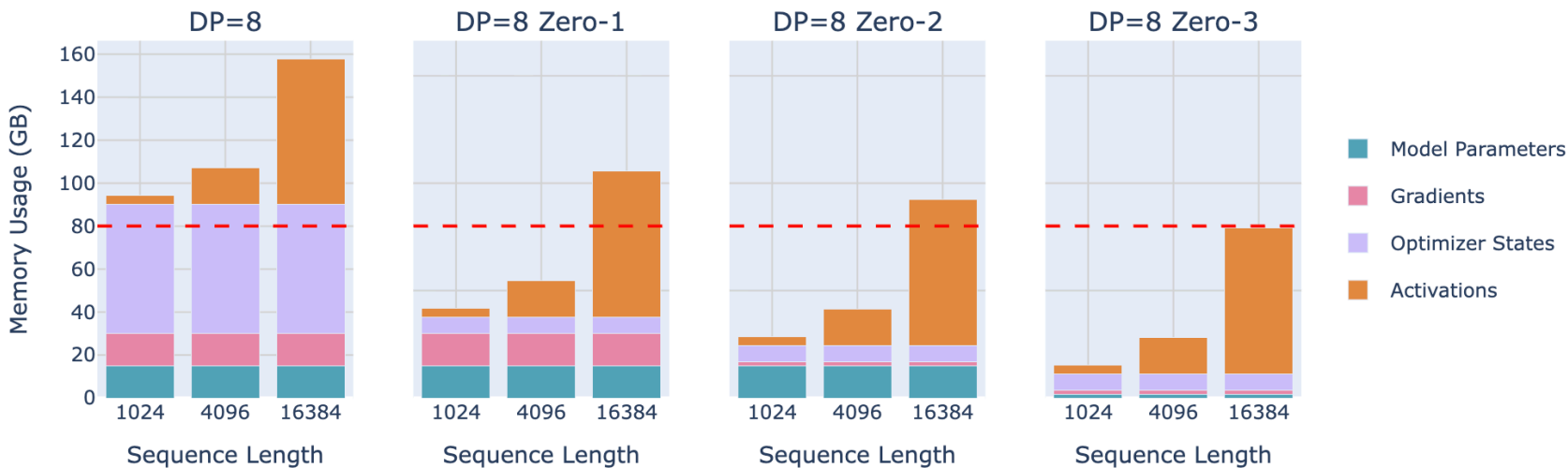
```
weights = 2 * 0.85B * 2 bytes = 3.4 GB  
grads = weights
```

Total: $(3.4 \text{ GB} * 2 + 0.275 \text{ GB} * (1 + 2 + 2)) = 7.9 \text{ GB} = 10 \% \text{ of an } 80\text{GB GPU}$

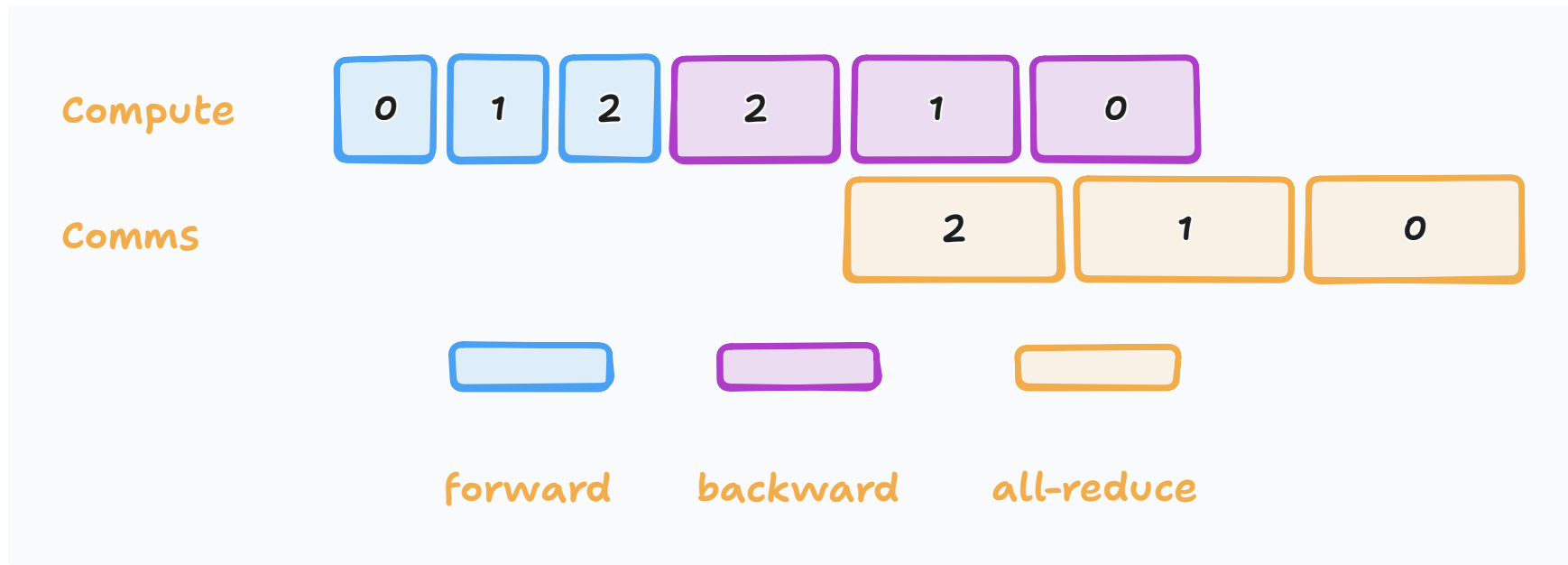
Great!

FSDP - memory usage

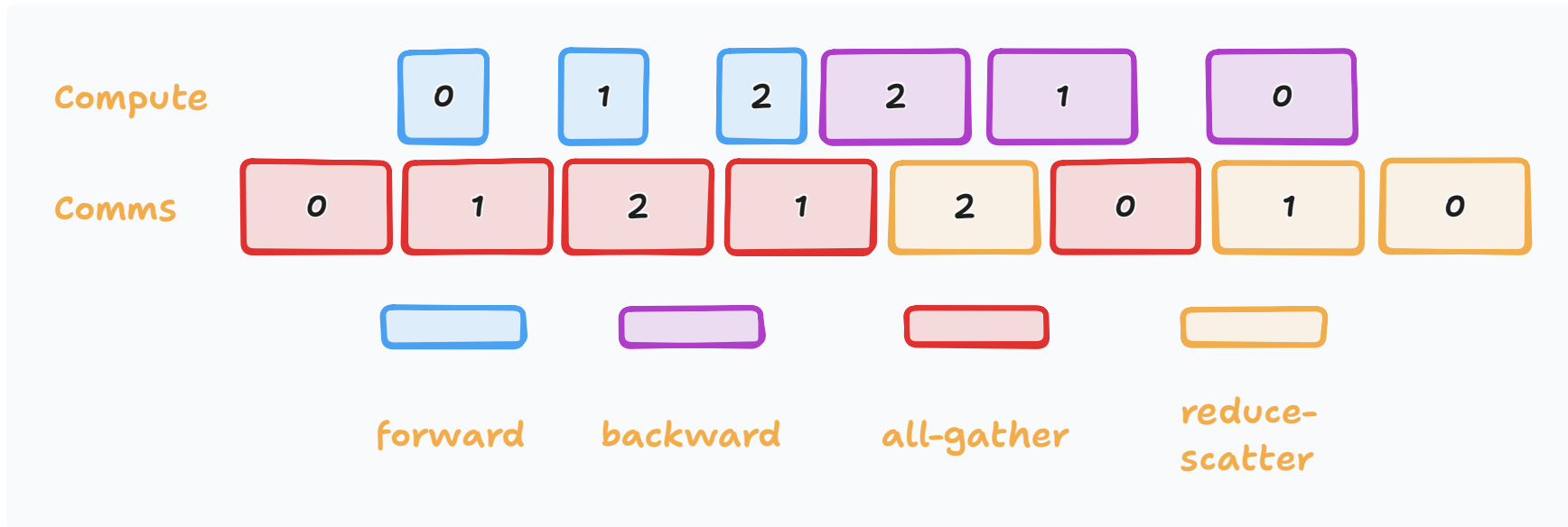
Memory Usage for 8B Model



Communication overlap - DDP



Communication overlap - FSDP



FSDP - communication overhead

ZERO-3 (FSDP) COMMUNICATION ANALYSIS

For ZeRO-3, parameters and gradients are sharded across GPUs. Let's analyze the communication pattern for a model with transformer blocks of size $16h^2$ parameters each:

- For each transformer block in the forward pass:
 - All-gather parameters: $16h^2 / DP$ bytes per rank
- For each transformer block in the backward pass:
 - All-gather parameters: $16h^2 / DP$ bytes per rank
 - Reduce-scatter gradients: $16h^2 / DP$ bytes per rank
- Total communication per block: $3 \cdot 16h^2 / DP$ bytes

FSDP - communication overhead

The communication time for all-gather operations is:

$$t_{comm} = 16h^2 \cdot \frac{DP - 1}{DP \cdot peak_bw}$$

The computation time for the forward pass of one decoder layer is:

$$t_{compute} = \frac{2 \cdot seq_len \cdot mbs \cdot (16 \cdot h^2)}{peak_flops} = \frac{32 \cdot seq_len \cdot mbs \cdot h^2}{peak_flops}$$

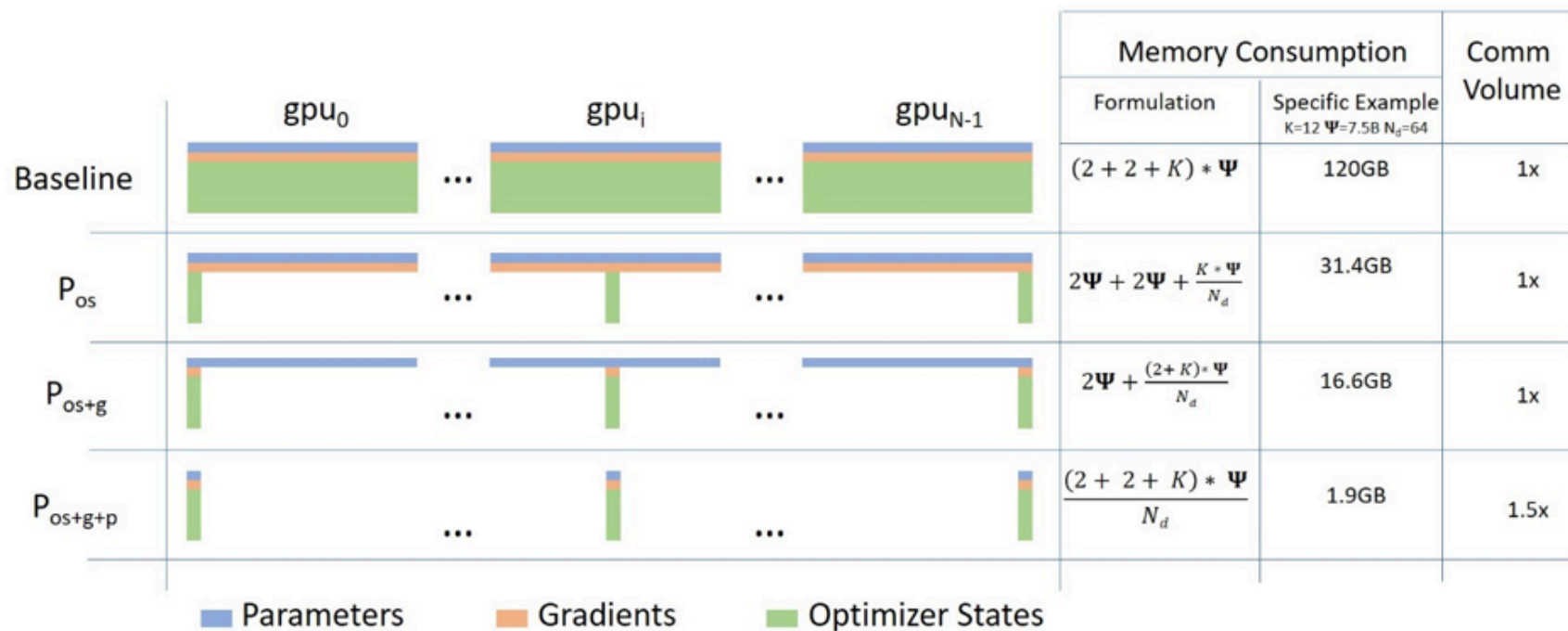
For effective overlap between computation and communication, we need:

$$\frac{t_{comm}}{t_{compute}} = \frac{1}{2 \cdot seq_len \cdot mbs} \cdot \frac{DP - 1}{DP} \cdot \frac{peak_flops}{peak_bw} \leq 1$$

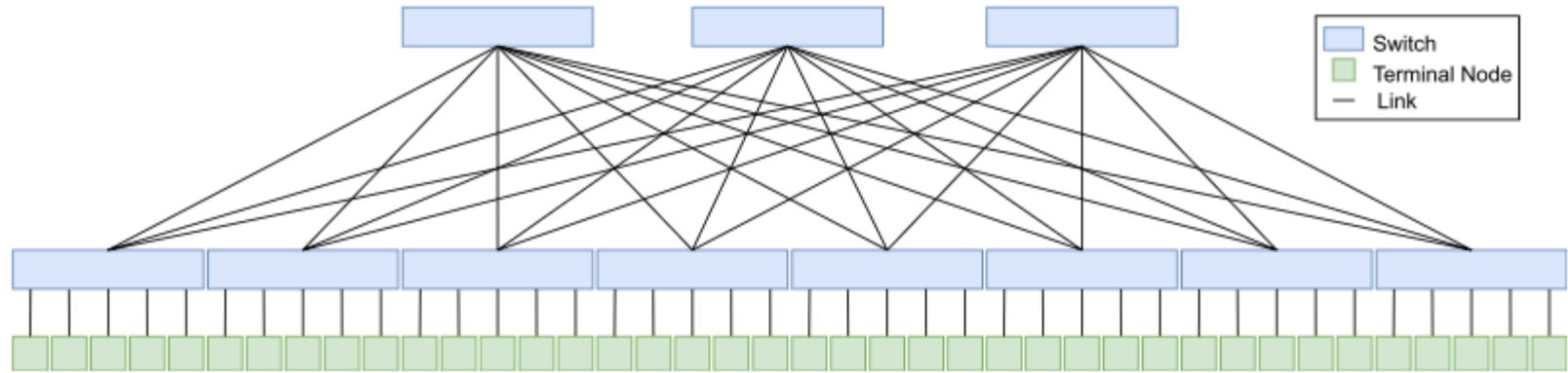
When this ratio is less than 1, the communication of parameters for the next layer can be hidden behind the computation of the current layer.

How to balance memory / comm
overhead

Sharding strategies - ZeRO



HSDP - Hybrid sharding (motivation)



(b) 40-node Slimmed Fat-tree (SFT).

Fig. 2 Example of different types of indirect networks

HSDP - Hybrid sharding

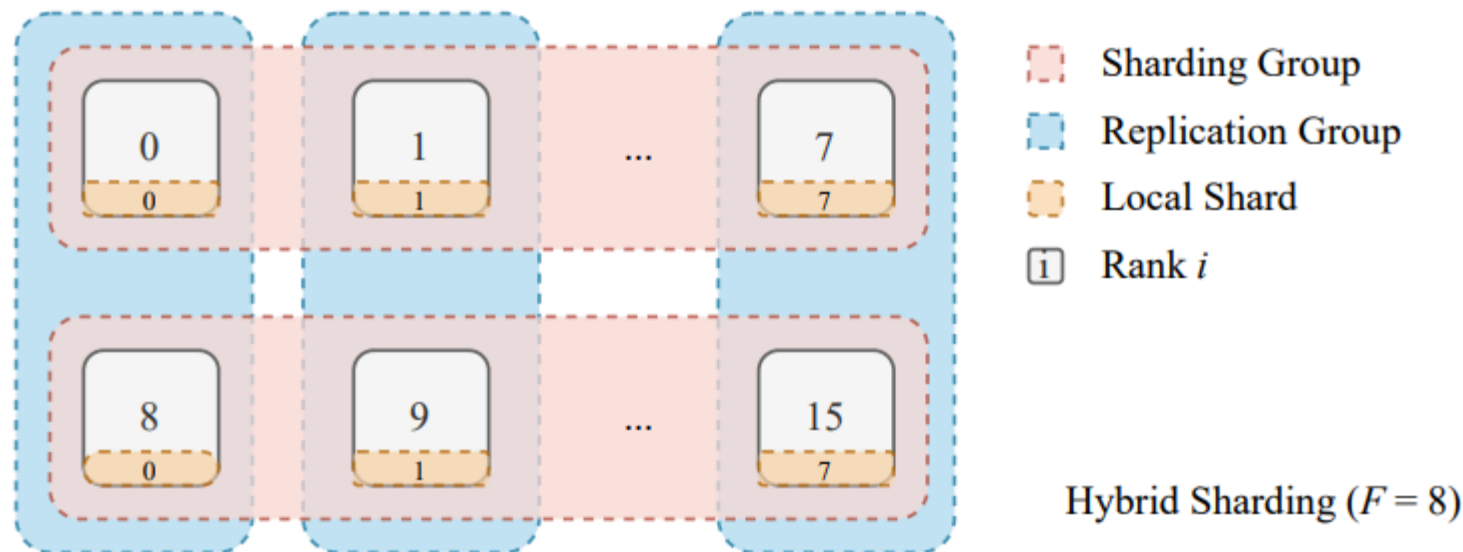
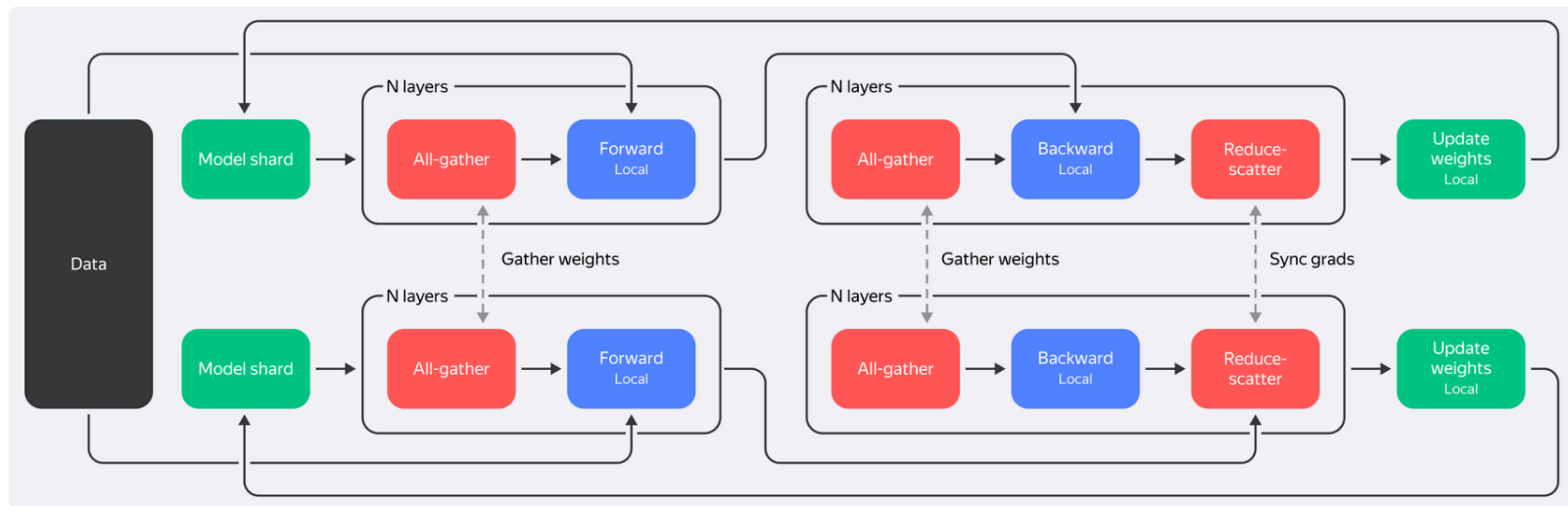


Figure 4: Hybrid Sharding on 16 GPUs: GPUs are configured into 2 sharding groups and 8 replication groups

Resharding to a smaller world size

In case there is enough memory post-forward resharding can be done to a smaller world size:

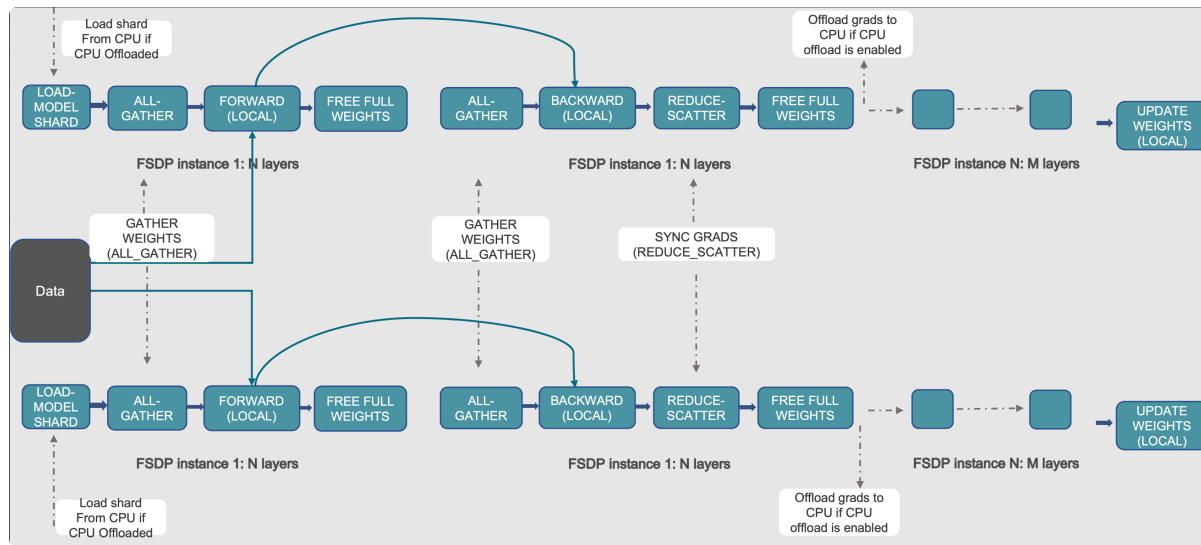
- less backward all-gather comms
- higher memory consumption



CPU offloading

Master weights, grads and optimizer states can be offloaded to CPU and only loaded to GPU before unsharding

- less memory consumption
- some overhead on CPU-GPU transfers and CPU optimizer step



FSDP implementations - FSDP1

FlatParameter, complex, bloated, buggy

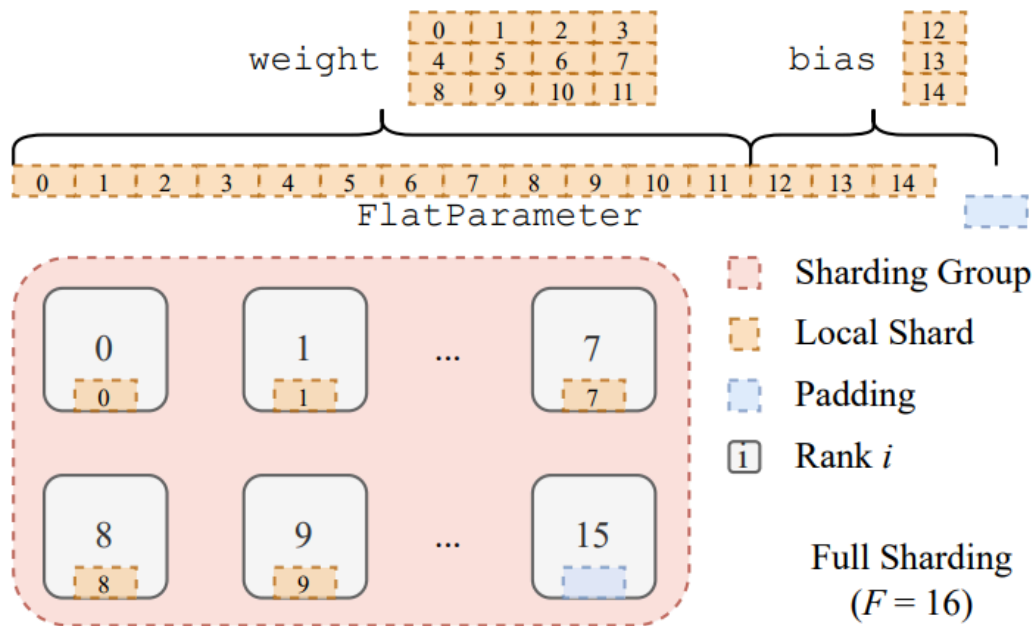


Figure 3: Full Sharding Across 16 GPUs

FSDP implementations

FSDP2

Per parameter sharding, clean, performant

YaFSDP

FSDP2 rewrite with flat sharding (and so less overhead)

SimpleFSDP

Compile native implementation with compile-time bucketing

Deepspeed-ZeRO

The og one, but not so performant

Recap

FSDP is a data parallelism strategy with parameter sharding which:

- reduces memory consumption (a lot compared to DDP)
- has 1.5 times more communications compared to DDP
- but (ideally) has no performance overhead
- is used in (almost) every large-scale distributed training recipe

We are hiring!

DM @lovanto on Telegram for details